

Reviewer Pack — Multiplicity of Symmetric Powers in Permanent Tensor Decompo...

Entry d4ccd48216c9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 13:05:11 UTC

Multiplicity of Symmetric Powers in Permanent Tensor Decomposition

Entry ID: d4ccd48216c9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 13:05:11 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Algebraic Groups **Field B** (complexity object): Boolean Circuit Complexity of Permanent

Statement:

For every $n \geq 2$, the multiplicity of the trivial representation in the decomposition of the symmetric power $S^n(\text{perm}_n)$ is $\Theta(n^2)$, where perm_n is the permanent polynomial of size n . This multiplicity equals the minimal number of gates in a depth-3 circuit computing perm_n over $\text{GF}(2)$.

Rationale (proposer's reasoning):

Symmetric power decompositions encode how tensor products of polynomials interact with algebraic group actions. The permanent's asymmetric representation-theoretic structure could reveal inherent complexity barriers via nontrivial multiplicities, linking geometric invariants to circuit size.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c20410ebf3688af0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        pivot = A[i][i]
        for j in range(i, n):
            A[i][j] /= pivot
        b[i] /= pivot
        for j in range(n):
            if i != j:
                factor = A[j][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]

def is_independent_system(A, b):
    gaussian_elimination(A, b)
    for i in range(len(b)):
        if abs(b[i]) > 1e-9:
            return False
    return True

def trivial_representation_multiplicity(P, n):
    A = [[0] * (n+1) for _ in range(n+1)]
    b = [0] * (n+1)
    for i in range(1, n+1):
        for j in range(1, n+1):
            if P[i-1][j-1] == 1:
                A[i][j] = 1
```

```

        b[j] += 1
    return is_independent_system(A, b)

def perm_n(n):
    def perm(a, i, j):
        a[i], a[j] = a[j], a[i]

    def generate_perms(a, start, end):
        if start == end:
            yield list(a)
        else:
            for i in range(start, end+1):
                perm(a, start, i)
                yield from generate_perms(a, start+1, end)
                perm(a, start, i)

    return list(generate_perms(list(range(1, n+1)), 0, n-1))

def symmetric_power(P, n):
    result = [[0] * n for _ in range(n)]
    perms = perm_n(n)
    for p in perms:
        for i in range(n):
            for j in range(n):
                if P[i][p[j]-1] == 1 and P[p[i]-1][j] == 1:
                    result[i][j] += 1
    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    P = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    S_n_P = symmetric_power(P, n)

    try:
        multiplicity = trivial_representation_multiplicity(S_n_P, n)
    except Exception as e:
        return {
            "metric_name": "trivial_representation_multiplicity",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": str(e)
        }

    # Placeholder for minimal depth-3 circuit size calculation
    min_circuit_size = n * (n - 1) // 2

    return {
        "metric_name": "trivial_representation_multiplicity",
        "metric_value": multiplicity,
        "instances_tested": 1,
        "conjecture_holds": multiplicity == min_circuit_size,
        "counterexample": ""
    }
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_multiplicity = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_multiplicity = math.sqrt(sum((r["metric_value"] - mean_multiplicity) ** 2 for r in results) /
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_multiplicity} std={std_multiplicity} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"multiplicity does not match circuit size\" first_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing verification of conjecture. | next: Run test with extended time limit and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109337
2	preregistration	ollama_remote	qwen3:8b	0	0	24202
3	novelty	ollama_remote	qwen3:8b	0	0	20788
4	novelty	ollama_remote	qwen3:8b	0	0	16822
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16744

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12917
7	judge	ollama_remote	qwen3:8b	0	0	65065

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 265875 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/d4ccd48216c9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d4ccd48216c9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d4ccd48216c9.tar.gz (if generated)